ELEC 475 – Lab 2: From LeNet-5 to AlexNet

Post-Lab Report

Aytu Tuna – Student number 20354469

ELEC 475, Fall 2026 (individual submission, no partner)

1. Architectural Specifications

All models take a 3 × 64 × 64 image. Output sizes use out = floor((in + 2p − k) / s) + 1. A layer's parameters are k·k·C_in·C_out + C_out for a convolution and in·out + out for a linear layer. The receptive field (RF) grows by (k − 1) × jump at each layer, where jump is the product of the strides so far. Parameter counts below are for 10 classes.

1.1 Base LeNet-5 (models/lenet.py)

Layer	Output	Parameters	RF
Conv 5×5, 6 filters, s=1, p=0 + Tanh	6 × 60 × 60	456	5
AvgPool 2×2, s=2	6 × 30 × 30	0	6
Conv 5×5, 16 filters, s=1, p=0 + Tanh	16 × 26 × 26	2,416	14
AvgPool 2×2, s=2	16 × 13 × 13	0	16
Flatten	2,704	0	
Linear 2,704 → 120 + Tanh	120	324,600	
Linear 120 → 84 + Tanh	84	10,164	
Linear 84 → 10	10	850	
Total		338,486	

Table 1: Base LeNet-5.

1.2 Modernized LeNet (models/lenet_modern.py)

Same layout, but Tanh is replaced by ReLU, average pooling by overlapping max pooling (3×3, stride 2), and the convolutions use padding 2.

Layer	Output	Parameters	RF
Conv 5×5, 6 filters, s=1, p=2 + ReLU	6 × 64 × 64	456	5
MaxPool 3×3, s=2	6 × 31 × 31	0	7
Conv 5×5, 16 filters, s=1, p=2 + ReLU	16 × 31 × 31	2,416	15
MaxPool 3×3, s=2	16 × 15 × 15	0	19
Flatten	3,600	0	
Linear 3,600 → 120 + ReLU	120	432,120	
Linear 120 → 84 + ReLU	84	10,164	
Linear 84 → 10	10	850	
Total		446,006	

Table 2: Modernized LeNet.

1.3 Full AlexNet-64 (models/alexnet.py)

Five convolution stages and a three-layer classifier with Dropout (p = 0.5). Each stage is Conv → Norm → ReLU. Norm depends on --norm: none (nothing), lrn (LocalResponseNorm(size=5, alpha=1e-4, beta=0.75, k=2) after Conv1 and Conv2 only) or bn (BatchNorm2d after every convolution).

Layer	Output	Parameters	RF
Conv1 5×5, 64 filters, s=2, p=2 + Norm + ReLU	64 × 32 × 32	4,864	5
MaxPool 3×3, s=2	64 × 15 × 15	0	9
Conv2 5×5, 192 filters, s=1, p=2 + Norm + ReLU	192 × 15 × 15	307,392	25
MaxPool 3×3, s=2	192 × 7 × 7	0	33
Conv3 3×3, 384 filters, s=1, p=1 + Norm + ReLU	384 × 7 × 7	663,936	49
Conv4 3×3, 384 filters, s=1, p=1 + Norm + ReLU	384 × 7 × 7	1,327,488	65
Conv5 3×3, 256 filters, s=1, p=1 + Norm + ReLU	256 × 7 × 7	884,992	81
MaxPool 3×3, s=2	256 × 3 × 3	0	97
Flatten	2,304	0	
Dropout 0.5, Linear 2,304 → 1,024 + ReLU	1,024	2,360,320	
Dropout 0.5, Linear 1,024 → 1,024 + ReLU	1,024	1,049,600	
Linear 1,024 → 10	10	10,250	
Total (none / lrn)		6,608,842	
BatchNorm, 2 per channel: 2 × (64+192+384+384+256)		2,560	
Total (bn)		6,611,402	

Table 3: AlexNet-64. After Conv5 the receptive field (81) is already larger than the 64-pixel image.

The handout gives about 6.3 M parameters and 250 MFLOPs for this model. The layer list in the handout adds up to 6.61 M, as shown above, and the provided profiler (thop, FLOPs = 2 × MACs) gives 438 MFLOPs. I kept the layers exactly as specified and report the measured values. The same profiler gives 7.17 and 9.19 MFLOPs for the two LeNets and 188.20 MFLOPs for the Zoo AlexNet (the handout lists about 4.8, 8.2 and 310–315). All FLOPs in this report are the values printed by compare_models.py (Listing 1).

1.4 PyTorch Model Zoo AlexNet (models/zoo.py)

torchvision.models.alexnet with the last layer replaced by Linear(4096, num_classes). It is built for 224 × 224 images. With a 64 × 64 input the feature map is only 1 × 1 after the last pooling, and AdaptiveAvgPool2d copies it to 6 × 6.

Layer	Output	Parameters	RF
Conv1 11×11, 64 filters, s=4, p=2 + ReLU	64 × 15 × 15	23,296	11
MaxPool 3×3, s=2	64 × 7 × 7	0	19
Conv2 5×5, 192 filters, p=2 + ReLU	192 × 7 × 7	307,392	51
MaxPool 3×3, s=2	192 × 3 × 3	0	67
Conv3 3×3, 384 filters, p=1 + ReLU	384 × 3 × 3	663,936	99
Conv4 3×3, 256 filters, p=1 + ReLU	256 × 3 × 3	884,992	131
Conv5 3×3, 256 filters, p=1 + ReLU	256 × 3 × 3	590,080	163
MaxPool 3×3, s=2	256 × 1 × 1	0	195
AdaptiveAvgPool to 6 × 6, Flatten	9,216	0	
Dropout 0.5, Linear 9,216 → 4,096 + ReLU	4,096	37,752,832	
Dropout 0.5, Linear 4,096 → 4,096 + ReLU	4,096	16,781,312	
Linear 4,096 → 10	10	40,970	
Total		57,044,810	

Table 4: PyTorch Zoo AlexNet with a 64 × 64 input.

2. Training Setup

All runs use train.py with the Adam optimizer (learning rate 1e-3, weight decay 1e-4), cross-entropy loss, batch size 128 and seed 42 (utils/seed.py). Images are resized to 64 × 64 and normalized. The GPU is an RTX 5070 Ti Laptop with PyTorch 2.14. LeNet models are trained for 5 epochs (as in the in-lab), AlexNet models for 10, and the two augmentation runs for 15. Rows marked "None" for augmentation were run with --no-augment. The exact commands are in commands.txt. Reported accuracy and Macro F1 are the best validation values over the epochs.

Besides the TODO sections, I made five changes to the provided code. (1) utils/dataset.py used the Tiny ImageNet training folder as the validation set. I changed it to the real validation split (10,000 images, val_annotations.txt). (2) I added a --history-file option to train.py that saves the per-epoch numbers. (3) I added a --device option to compare_models.py. (4) I fixed evaluate.py so that it can load the saved checkpoints and the zoo_alexnet model. (5) I added the --plot-ablation command to utils/visualization.py.

3. Empirical Results & Figures

3.1 Master ablation tables

Model	Conv depth	Normalization & regularization	Data aug.	Params (M)	FLOPs (M)	Mean epoch time (s)	Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	19.7	0.23
2. Modernized LeNet	2	None (ReLU + overlapping MaxPool)	None	0.446	9.19	18.9	0.20
3. AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.70	25.3	0.46
4. AlexNet + LRN	5	LRN + Dropout (p=0.5)	None	6.609	436.70	27.1	0.72
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	24.7	0.64
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p=0.5)	PyTorch aug.	6.611	437.97	27.4	0.64
7. PyTorch Zoo AlexNet	5	Dropout (p=0.5), AdaptiveAvgPool2d	None	57.045	188.20	26.6	1.00

Table 7.3a (handout numbering): Configuration and complexity (CIFAR-10, 10 classes). Epoch time is the mean over all epochs and includes the validation pass. For rows 3–5 it is also averaged over the three seeds. Latency is for one image on the GPU (compare_models.py).

Model	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	54.91	54.73	9.94	7.83
2. Modernized LeNet	65.40	65.33	19.16	16.69
3. AlexNet (No Norm)	74.58	74.66	17.04	13.98
4. AlexNet + LRN	74.08	74.04	16.99	14.40
5. AlexNet + BatchNorm	77.02	77.03	30.54	28.63
6. AlexNet + BN + Aug	78.90	78.95	30.46	28.11
7. PyTorch Zoo AlexNet	67.11	66.90	11.85	9.17

Table 7.3b (handout numbering): Best validation accuracy and Macro F1 (seed 42). Rows 1–2: 5 epochs. Rows 3–7: 10 epochs. Row 6 on CIFAR-10 is read from the first 10 epochs of the 15-epoch augmented run so that it can be compared with row 5.

3.2 Multi-seed results (normalization study)

AlexNet variant (CIFAR-10, 10 epochs, no aug.)	Val Acc, seeds 42 / 123 / 999	Val Acc Mean ± Std (%)	Macro F1 Mean ± Std (%)
No Norm	74.58 / 75.25 / 73.80	74.54 ± 0.73	74.57 ± 0.63
LRN	74.08 / 73.72 / 73.67	73.82 ± 0.22	73.67 ± 0.32
BatchNorm	77.02 / 79.10 / 79.92	78.68 ± 1.49	78.63 ± 1.54

Table 5: Three seeds per variant, reported as mean ± sample standard deviation (utils/seed.py).

3.3 Extended ablation exploration

Experiment (CIFAR-10)	Epochs	Val Acc (%)	Macro F1 (%)	Δ Val Acc
Baseline: AlexNet + BN, lr 1e-3, batch 128	10	77.02	77.03	–
Learning rate 3e-4	10	80.98	81.26	+3.96 (vs baseline)
Batch size 256	10	79.33	79.47	+2.31 (vs baseline)
15 epochs, no augmentation	15	81.65	81.37	+4.63 (vs baseline)
15 epochs, with augmentation	15	82.69	82.76	+1.04 (vs 15 ep. no aug.)
Zoo AlexNet, random weights	10	67.11	66.90	-9.91 (vs baseline)
Zoo AlexNet, ImageNet-pretrained (--pretrained)	10	77.43	77.28	+10.32 (vs Zoo random)

Table 6: Extended ablation. All runs use seed 42 and no augmentation unless stated. The seed-42 baseline was the lowest of the three seeds in Table 5. Against the 3-seed mean (78.68) the deltas are +2.30 (learning rate 3e-4) and +0.65 (batch size 256), so the batch-size result is inside the seed range.

3.4 Generalization gap (augmentation study)

AlexNet + BN, 15 epochs	Train Acc (%)	Val Acc (%)	Gap Δ = Train − Val	Best Val Acc (%)	Best Macro F1 (%)
Without augmentation	90.50	80.30	10.20	81.65	81.37
With augmentation	84.53	82.69	1.84	82.69	82.76

Table 7: Train accuracy, validation accuracy and gap at the last epoch (15).

3.5 Training and validation curves

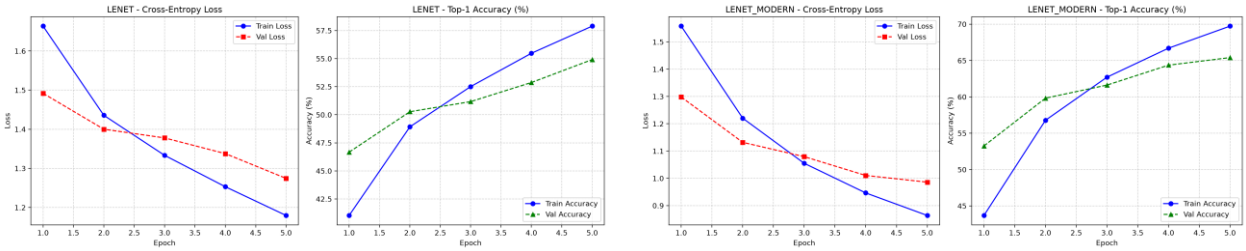
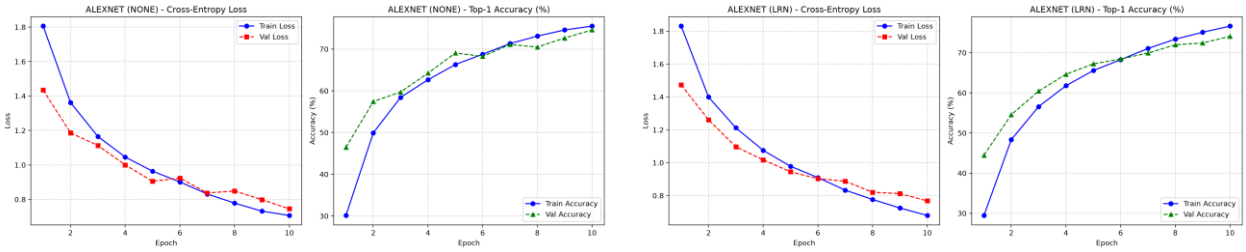


Figure 1: Base LeNet-5 (left) and Modernized LeNet (right), CIFAR-10, 5 epochs.



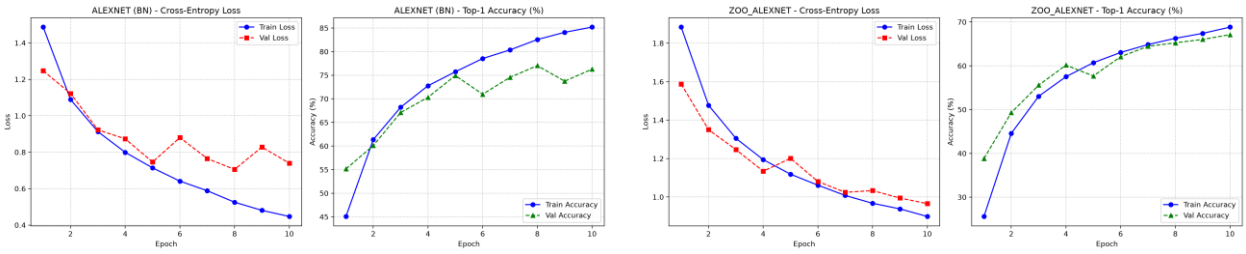


Figure 2: AlexNet with no normalization, LRN, BatchNorm, and the Zoo AlexNet, CIFAR-10, 10 epochs.

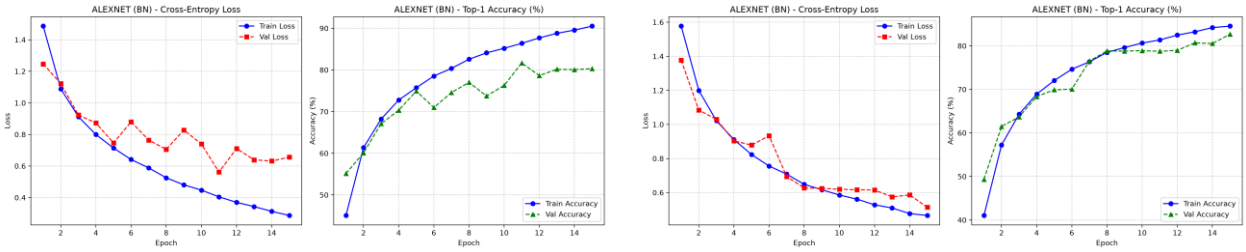


Figure 3: AlexNet + BN without (left) and with (right) data augmentation, CIFAR-10, 15 epochs.

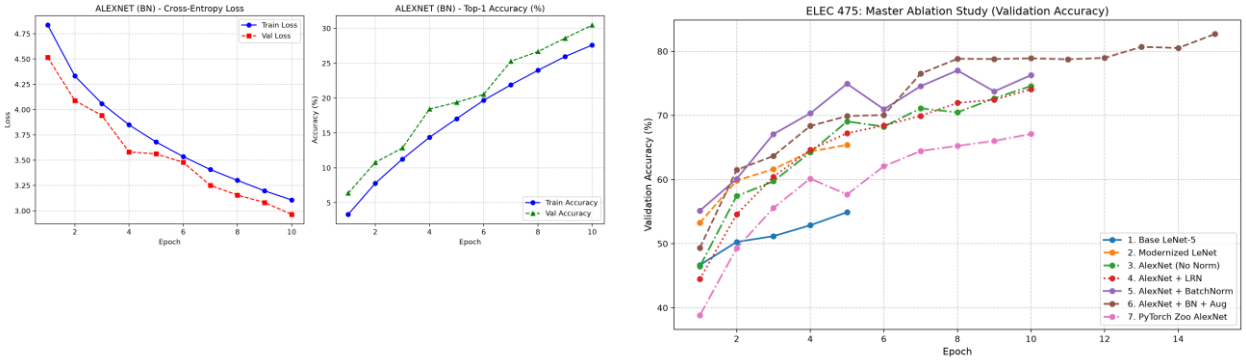


Figure 4: AlexNet + BN + augmentation on Tiny ImageNet (left). Validation accuracy of all seven table rows on CIFAR-10 (right, utils.visualization --plot-ablation).

3.6 Output of compare_models.py

Model Architecture	Params (M)	MACs (M)	FLOPs (M)	Latency (ms)	Throughput (img/s)
1. LeNet-5 (Base)	0.338	3.59	7.17	0.23	4415.0
2. LeNet-Modern (ReLU+Pool)	0.446	4.59	9.19	0.20	4916.4
3. AlexNet-64 (None)	6.609	218.35	436.70	0.46	2190.1
4. AlexNet-64 (LRN)	6.609	218.35	436.70	0.72	1397.8
5. AlexNet-64 (BatchNorm)	6.611	218.98	437.97	0.64	1565.7
6. Torchvision Zoo AlexNet	57.045	94.10	188.20	1.00	1003.2

Listing 1: python compare_models.py --dataset cifar10 --device cuda

3.7 Output of demo_transforms.py

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================


[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
```

```
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :   12.63 ms  ( 79,165 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   20.05 ms  ( 49,870 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  202.97 ms  (  4,927 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Listing 2: python demo_transforms.py

3.8 Dataset figures from utils/eda.py

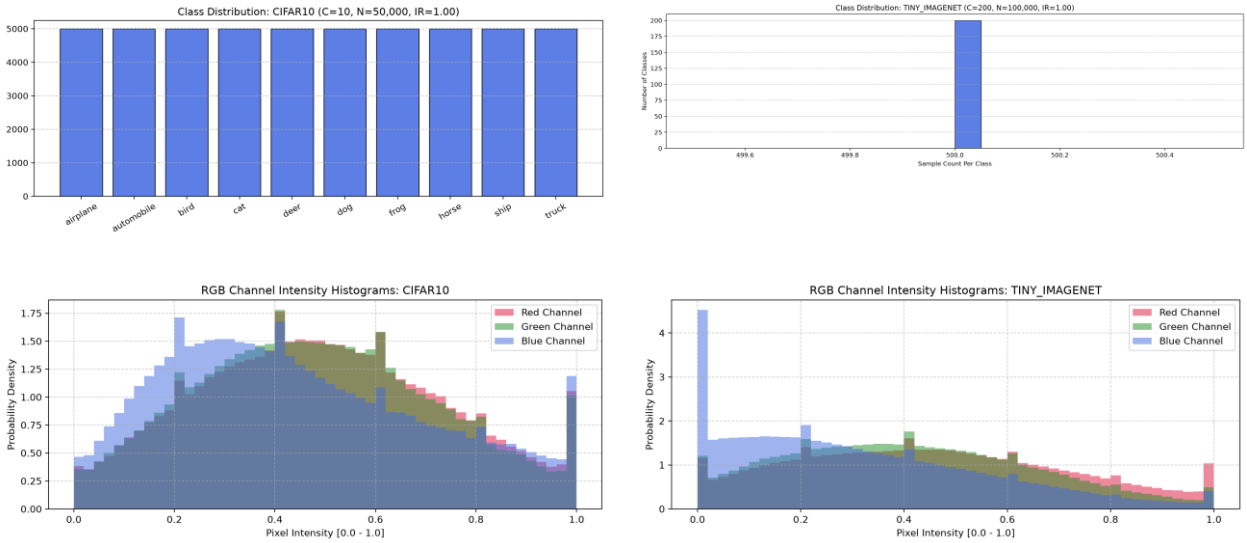


Figure 5: Class balance (top) and RGB channel distributions (bottom) for CIFAR-10 (left) and Tiny ImageNet (right). Both datasets are balanced.

4. Technical Analysis & Discussion

4.1 LeNet to AlexNet

Replacing Tanh and average pooling with ReLU and overlapping max pooling raised CIFAR-10 accuracy from 54.9% to 65.4% in the same 5 epochs, and from 9.9% to 19.2% on Tiny ImageNet. ReLU does not saturate, so the gradients stay larger and the model learns faster. AlexNet-64 has about 15 times more parameters and 48 times more FLOPs than the modernized LeNet and reaches 74.6% on CIFAR-10 without normalization. The extra depth and channels explain the gain. The receptive field of AlexNet-64

covers the whole image, while the modernized LeNet only sees a 19-pixel window. On Tiny ImageNet the deeper model does not help by itself: AlexNet without normalization (17.0%) and with LRN (17.0%) are below the modernized LeNet (19.2%), even with 10 epochs against 5. Their training accuracy is still only about 15% after 10 epochs, so they are learning slowly, not overfitting. Only with BatchNorm does the deeper model pay off (30.5%).

4.2 Normalization study

Performance: Yes, BatchNorm gave the best accuracy and Macro F1. Over three seeds it reached 78.68 ± 1.49%. No normalization reached 74.54 ± 0.73% and LRN 73.82 ± 0.22% (Table 5). Macro F1 follows the same order. The difference is larger than the spread between seeds. It is bigger on Tiny ImageNet: 30.5% with BatchNorm against 17.0% (none) and 17.0% (LRN). LRN gave no gain over no normalization.

Why: BatchNorm keeps the inputs of each layer at the same mean and variance during training. Later layers do not have to keep adjusting to changes in the earlier layers (less internal covariate shift). The gradients also depend less on the scale of the weights, which makes the loss surface smoother, so the same learning rate makes faster progress. The training loss shows this. After 10 epochs it is 0.45 with BatchNorm, 0.71 with no normalization and 0.68 with LRN. LRN only scales a channel by its neighbouring channels. It does not control the mean or variance, so it does not help in the same way.

Computational overhead: FLOPs: No. BatchNorm adds 2,560 parameters and 1.27 MFLOPs (0.3%), and the profiler counts no FLOPs for LRN. Time: Yes for LRN, No for BatchNorm. Mean epoch time was 25.3 s (none), 27.1 s (LRN) and 24.7 s (BatchNorm). Repeated runs of the same model differ by up to about 3 s (the three no-norm runs took 23.2, 26.3 and 26.4 s) because data loading takes most of the epoch, so none and BatchNorm are the same within noise and only LRN is slower. Single-image latency was 0.46 ms (none), 0.72 ms (LRN) and 0.64 ms (BatchNorm). LRN was the slowest and did not improve accuracy. BatchNorm cost almost nothing in training time and gave about 4 points of accuracy.

Stability: BatchNorm had the most stable training. The training loss falls at every epoch for all three variants, but with BatchNorm it falls fastest and is almost the same in all three seeds (0.45 / 0.45 / 0.44 after 10 epochs, against 0.71 / 0.60 / 0.62 with no normalization and 0.68 / 0.68 / 0.66 with LRN). The validation curve of BatchNorm is the least smooth. Validation accuracy jumps by several points between epochs (Figure 2), and in one seed it dropped to 57.35% for one epoch and then recovered. The validation curves of none and LRN are smoother but slower. A lower learning rate helped accuracy: with 3e-4 the same model reached 80.98% (Table 6). It did not remove the jumps (77.5 → 74.5 → 79.5% in epochs 5 to 7). A likely cause is that validation uses BatchNorm's running mean and variance, which lag behind while the weights change quickly.

4.3 Data augmentation

Generalization gap: Yes, augmentation made the gap smaller. Without augmentation the model ended at 90.50% train and 80.30% validation accuracy (Δ = 10.20). With augmentation it ended at 84.53% and 82.69% (Δ = 1.84).

Validation metrics: Yes, by a small amount. Best validation accuracy went from 81.65% to 82.69% and Macro F1 from 81.37% to 82.76%. Random crops, flips and colour changes do not change the label, but the network never sees exactly the same image twice. It cannot memorise pixel positions or exact

colours, so the filters have to respond to the object itself. Without augmentation the validation loss stops improving after about epoch 11 while the training loss keeps falling (Figure 3, left). With augmentation both keep falling together and validation accuracy is still rising at epoch 15, so longer training would likely widen the difference.

Note: training accuracy is measured during training, with Dropout on. In the augmented run it is also measured on augmented images. This makes training accuracy look a little lower, so the real gaps are slightly larger than the ones in Table 7.

4.4 Tiny ImageNet

Above chance: Yes. AlexNet + BN reached 30.46% validation accuracy (Macro F1 28.11%) after 10 epochs, about 60 times the 0.5% of random guessing.

Compared with CIFAR-10: much lower, 30.5% accuracy against 78.9% and 28.1% Macro F1 against 78.9% for the same model and number of epochs. There are three reasons. Tiny ImageNet has 200 classes instead of 10, and many are similar (for example several dog breeds). It has only 500 training images per class instead of 5,000. Its images are real 64 × 64 photos with busy backgrounds, while CIFAR-10 images are 32 × 32 pictures of one centred object, enlarged to 64 × 64.

Overfitting trajectory: No. On Tiny ImageNet the validation loss was at its lowest at epoch 10 (2.96 with augmentation, 2.98 without) and training accuracy had not passed validation accuracy (27.6% against 30.5% with augmentation, 30.2% against 30.5% without). So the model is still under-trained, not overfitting (Figure 4, left). On CIFAR-10 without augmentation the validation loss becomes noisy from about epoch 5 and stops improving after epoch 11 (0.56), while the training loss keeps falling. Training accuracy is already 8.9 points above validation accuracy at epoch 10. The harder task needs more epochs before overfitting appears. For the same reason augmentation did not help on Tiny ImageNet within 10 epochs (30.46% with, 30.54% without).

4.5 PyTorch Zoo AlexNet

Trained from random weights, the Zoo AlexNet was worse than my AlexNet-64 on both datasets (67.1% on CIFAR-10, 11.9% on Tiny ImageNet) although it has 57 M parameters. Its 11 × 11, stride-4 first layer is made for 224-pixel images. On a 64-pixel image the feature map shrinks to 1 × 1, so most spatial detail is lost. The model also has no normalization. The small feature maps are also the reason it has fewer FLOPs (188 M) than AlexNet-64 (438 M): its convolutions work on maps of 15 × 15 down to 3 × 3. About 96% of its parameters are in the linear layers, where each weight is used only once per image. With ImageNet-pretrained weights it reached 77.4% on CIFAR-10, +10.3 points over random weights, which shows the value of pretrained features.

4.6 Limits

Most results come from one seed and short training (5 to 15 epochs). The BatchNorm seeds differ by a standard deviation of 1.49 points (Table 5), so differences of less than about two points may be due to chance. The reported numbers are the best epoch on the validation set. There is no separate test set, so they are slightly optimistic.